the cache, and only update the DB if another client requests the same entry; or, on the other hand, you could decide to always update the DB.

Even more, you could decide to write on the DB only and update the cache just when data is requested for read, or on the contrary update the cache on writes.

These *policies* described above all have names, since they are widely used in software design and engineering.

Write-Behind (or *write-back)*, our first example, is a storage policy where data is written into cache on every change, while it's written into the corresponding location on the main storage (memory, DB, etc…) only at specified intervals of time, or under certain conditions (for instance, on reads).

In this case the data on cache is always fresh, while data on the DB (or other support) might be stale. This policy helps keeping latency low and also reducing the load on the DB, but it might lead to data loss, for instance when we write back only on reads, and an entry stored in cache is never read after write. In some applications this data loss is fine, and in those cases write back is the preferred policy.

Write-Through (or *write-ahead*), writes entries always both on cache and on the main storage at the same time: this way the database will usually have only one write, and will (almost) never be hit by the application for read. This is slower than write-behind, but reduces the risk of data loss, bringing it down to zero – with only the exception of edge cases and malfunctions. Write-through strategy doesn't improve writing performance at all (but caching would still